Channel Partner Instructions: Compare

Module Features

- Responsive design with support for large, medium and small viewports
- Standard and high resolution images
- Locally hosted images
- Interactive product selection via dropdowns (interactive mode)
- Static product display (static mode)
- URL parameter-based product lineup configuration
- JSON-based content management

Viewport Definitions

The responsive design adapts to different screen sizes based on the following viewport ranges:

Viewport	Minimum	Maximum
small	320px	734px
medium	735px	1068px
large	1069px	∞

Minimum Width Requirements & Preventing Clipping

Global Minimum Width: 320px

The exported HTML content has a global minimum width of 320px. This is the standard minimum width for mobile phone displays.

Recommended Minimum Width: 380px

While the technical minimum is 320px, we recommend a minimum container width of **380px** to account for extra padding and prevent layout issues across different designs.

Important Implementation Guidelines:

- The exported HTML must be embedded in containers with a minimum width of **320px** for proper display
- **Recommended:** Use a minimum container width of **380px** to account for extra padding on different designs
- For optimal results without clipping, containers should be at least as wide as the viewport breakpoints listed above

Recommendation: Ensure your target container element has sufficient width to accommodate the content at each viewport size to prevent clipping and maintain proper layout.

File Structure

When you receive the exported Compare module, you will find the following files:

```
compare-module/
├── index.html                          # Main HTML file
├── scripts/
│   ├── channel-html-compare.json       # Compare data (products, features, structure)
│   └── channel-html-compare-content.json   # Localized content (i18n strings)
└── images/                             # Product images
```

File Descriptions

- **`index.html`** : The main HTML file containing the compare structure, styles, and JavaScript. This file loads and displays the compare data.

- **`channel-html-compare.json`** : Contains the structural data for the compare including products, categories, features, and configuration.
- **`channel-html-compare-content.json`** : Contains all localized text content using i18n keys (e.g., `__i18n_0__` , `__i18n_1__` , etc.).
- **`images/`** : Directory containing all referenced images.

How JSON Data Is Loaded

The `index.html` file contains a `data-compare-json-path` attribute on the main compare element that tells the JavaScript where to find the JSON files:

```
data-compare-json-path="./scripts/channel-html-compare.json"
```

The JavaScript automatically fetches both JSON files — the structure file from the path specified, and the content file by replacing `.json` with `-content.json` (e.g., `./scripts/channel-html-compare-content.json`).

If you host the JSON files at a different location, update the `data-compare-json-path` attribute to point to the new path.

Locating the Data Section in the HTML

Inside `index.html` , the JSON data section is clearly marked with comment boundaries:

```
<!-- BEGIN > CHANNEL SCRIPTS -->
  ...
<!-- END > CHANNEL SCRIPTS -->
```

This is the designated area where the compare module's data is managed. Look for these comments when you need to locate or modify the data configuration in the HTML file.

Working with JSON Files

The compare module uses two JSON files for content management:

Structure File (`channel-html-compare.json`)

This file contains:

- Product definitions and metadata
- Category structure
- Feature organization
- Image references
- Configuration settings (column order, max columns per viewport)

Content File (`channel-html-compare-content.json`)

This file contains all translatable text content:

```
{
  "__i18n_0__": "Product A",
  "__i18n_1__": "Starting at $XXX",
  "__i18n_2__": "Buy"
}
```

To edit text content:

1. Open `channel-html-compare-content.json`
2. Find the i18n key you want to modify (e.g., `__i18n_0__`)
3. Update the value
4. Save the file

Important: Do not change the keys (e.g., `__i18n_0__`), only change the values. The structure file references these keys.

How the Files Work Together

The structure file references content using i18n keys:

```
{
  "value": "__i18n_0__"
}
```

When the page loads, the JavaScript replaces `__i18n_0__` with the actual text from the content file.

Embedding JSON Data Directly in HTML (Alternative to Separate Files)

By default, the compare module fetches its JSON data from separate files in the `scripts/` directory. If you are unable to host the JSON files separately (e.g., due to hosting restrictions or CORS limitations), you can embed the JSON data directly inside the `index.html` file.

How to Embed the JSON Data

1. Open `index.html` in your text editor
2. Locate the data section marked by the comment boundaries:

   ```
   <!-- BEGIN > CHANNEL SCRIPTS -->
     ...
   <!-- END > CHANNEL SCRIPTS -->
   ```

3. Add two `<script>` tags with `type="application/json"` between the comment boundaries — one for the structure data and one for the content data:

   ```
   <!-- BEGIN > CHANNEL SCRIPTS -->
   <script type="application/json" id="channel-html-compare">
     { ... paste the contents of channel-html-compare.json here ... }
   </script>
   <script type="application/json" id="channel-html-compare-content">
     { ... paste the contents of channel-html-compare-content.json here ... }
   </script>
   <!-- END > CHANNEL SCRIPTS -->
   ```

Important Requirements

- The `type` attribute **must** be `application/json` (not `text/javascript`)
- The `id` attribute **must** match the JSON filename without the `.json` extension:
  - Structure data: `id="channel-html-compare"`
  - Content data: `id="channel-html-compare-content"`
- The JSON content must be valid JSON — use a JSON validator if you encounter issues
- When JSON data is embedded inline, the module will use it directly and skip fetching from external files

When to Use This Approach

- Your hosting environment does not allow serving separate `.json` files
- Cross-origin restrictions prevent the JavaScript from fetching the JSON files
- You prefer a single-file deployment with no external dependencies (aside from images)

Viewport Behavior

The compare module displays different numbers of products based on the viewport size. This is controlled by the `initialColumnState` in the JSON structure file:

```
{
  "initialColumnState": {
    "small": [0, 1],      // Shows products 0 and 1 on small screens
    "medium": [0, 1, 2],  // Shows products 0, 1, and 2 on medium screens
    "large": [0, 1, 2, 3] // Shows products 0, 1, 2, and 3 on large screens
  }
}
```

Product indexes are 0-based:

- First product = 0
- Second product = 1
- Third product = 2
- etc.

Changing Which Products Display Per Viewport

To change which products appear at each viewport:

1. Open `channel-html-compare.json`
2. Find the `initialColumnState` object
3. Modify the arrays for each viewport:

```
{
  "initialColumnState": {
    "small": [0, 2],      // Show first and third products
    "medium": [0, 1, 3],  // Show first, second, and fourth products
    "large": [0, 1, 2, 3] // Show first four products
  }
}
```

Removing Products

To remove a product from the Compare, you only need to delete it from the JSON and update references. **You do not need to renumber the remaining product indexes.**

Important: Each product object contains all of its category data nested inside it. When you delete a product, all its associated category data is automatically removed.

Step 1: Remove from Products Object

1. Open `channel-html-compare.json`
2. Find the `products` object
3. Delete the entire product entry (including all its nested category data)

Note: Products are keyed by their `en_WW` product name (e.g., "Product A", "Product B").

Example - Before:

```
{
  "products": {
    "Product A": {
      "productIndex": 0,
      "productName": "__i18n_2__",
      "categories": {
        "display": [...],
        "finish": [...],
        "camera": [...]
      }
    },
    "Product B": {
      "productIndex": 1,
      "productName": "__i18n_3__",
      "categories": {
        "display": [...],
        "finish": [...],
        "camera": [...]
      }
    },
    "Product C": {
      "productIndex": 2,
      "productName": "__i18n_4__",
      "categories": {
        "display": [...],
        "finish": [...],
        "camera": [...]
      }
    }
  }
}
```

Example - After (removing Product B):

```
{
  "products": {
    "Product A": {
      "productIndex": 0,
      "productName": "__i18n_2__",
      "categories": {
        "display": [...],
        "finish": [...],
        "camera": [...]
      }
    },
    "Product C": {
      "productIndex": 2,
      "productName": "__i18n_4__",
      "categories": {
        "display": [...],
        "finish": [...],
        "camera": [...]
      }
    }
  }
}
```

Note: The `productIndex` values remain unchanged (0 and 2). You don't need to renumber them. All category data for Product B is removed when you delete the product object.

Step 2: Update initialColumnState

Update the `initialColumnState` to remove references to the deleted product index:

Before (3 products: 0, 1, 2):

```
{
  "initialColumnState": {
    "small": [0, 1],
    "medium": [0, 1, 2],
    "large": [0, 1, 2]
  }
}
```

After (2 products: 0, 2):

```
{
  "initialColumnState": {
    "small": [0, 2],
    "medium": [0, 2],
    "large": [0, 2]
  }
}
```

Step 3: Update Option Groups (for Interactive Mode)

If using interactive mode with dropdowns, update the option groups:

1. Find `optionGroups` in the JSON
2. Remove the option corresponding to the deleted product index
3. Leave remaining product indexes unchanged

Example - Before:

```
{
  "optionGroups": [
    {
      "value": "Model",
      "options": [0, 1, 2]
    }
  ]
}
```

Example - After removing product 1:

```
{
  "optionGroups": [
    {
      "value": "Model",
      "options": [0, 2]  // ← Just remove 1, keep 0 and 2 as-is
    }
  ]
}
```

Step 4: Clean Up Unused i18n Content (Optional)

Optionally, remove unused i18n keys from `channel-html-compare-content.json` to keep the file clean.

Converting Between Static and Interactive Modes

The compare module supports two display modes:

- **Interactive Mode**: Users can select which products to compare using dropdown menus
- **Static Mode**: Products are fixed and cannot be changed by the user

Converting to Static Mode

1. Open `index.html` in your text editor
2. Find the interactive header row:

   ```
   <div id="interactive-compare-header" role="row" class="compare-row" compare-header-show="true">
   ```

3. Change `compare-header-show="true"` to `compare-header-hide="true"`:

   ```
   <div id="interactive-compare-header" role="row" class="compare-row" compare-header-hide="true">
   ```

4. Find the static header row:

```
<div id="static-compare-header" role="row" class="compare-row" data-column-group compare-header-
hide="true">
```

5. Change `compare-header-hide="true"` to `compare-header-show="true"` :

```
<div id="static-compare-header" role="row" class="compare-row" data-column-group compare-header-
show="true">
```

Converting to Interactive Mode

Follow the steps above in reverse:

- Set `compare-header-show="true"` on the **interactive** header
- Set `compare-header-hide="true"` on the **static** header

Updating Buy Button

Updating href

Updating all buttons

The buy button is by default configured to have an `href` of `#` for all buy buttons. If you wish to keep the styling as is and update the href of all the buttons to a different URL, you can do so by updating the corresponding i18n value.

1. Open the `channel-html-compare.json` file
2. Search for the `"buy"` section
3. Make note of the i18n number set for `value` ex: `__i18n_XX__`
4. Open the `channel-html-compare-content.json` file
5. Search for the i18n number and update the value ex: `"__i18n_XX__": "#",` to `"__i18n_XX__": "[YOUR URL]",`

Updating each button

If you would like to update each buy button by product, you can search for the Product within the `channel-html-compare.json` file and update the `value` to your desired href. EX:

```
        "buy": [
          {
            "template": "preview-item-button",
            "aria": "__i18n_X__",
            "value": "__i18n_XX__",
            "featureTitleOverride": "__i18n_XXXX__"
          }
        ]
```

```
        "buy": [
          {
            "template": "preview-item-button",
            "aria": "__i18n_X__",
            "value": "[YOUR URL]",
            "featureTitleOverride": "__i18n_XXXX__"
          }
        ]
```

NOTE: This will need to be done for every product's buy button

Updating buy button display text

Updating all buttons

The buy button is by defalut configured to have a default value of `Buy` for all buy buttons. If you wish to keep the styling as is and update the display text for all buttons you can do so by updating the corresponding i18n value.

1. Open the `channel-html-compare.json` file

2. Search for the `"buy"` section
3. Make note of the i18n number set for `featureTitleOverride` ex: `__i18n_X__`
4. Open the `channel-html-compare-content.json` file
5. Search for the i18n number and update the value you made a note of in step 3 ex: `"__i18n_XX__": "#",` to `"__i18n_XX__": "[YOUR URL]",`

Updating individual buttons

If you would like to update each buy button by product, you can search for the Product within the `channel-html-compare.json` file and update the `featureTitleOverride` property.

```
"buy": [
  {
    "template": "preview-item-button",
    "aria": "__i18n_X__",
    "value": "__i18n_XX__",
    "featureTitleOverride": "__i18n_XXXX__"
  }
]
```

```
"buy": [
  {
    "template": "preview-item-button",
    "aria": "__i18n_X__",
    "value": "__i18n_XX__",
    "featureTitleOverride": "[YOUR DISPLAY TEXT]"
  }
]
```

NOTE: You can make this change to only one button if needed. If the `featureTitleOverride` is not updated the default i18n thats set will be used.

URL Parameter Configuration

The compare module supports URL parameters to override the default product lineup. This allows you to create links that display specific product combinations.

Basic Usage

Add the `apple-compare` parameter to your URL with a comma-separated list of product numbers:

```
https://example.com/compare.html?apple-compare=1,3,2
```

This will display:

- Product 1 (second product, 0-indexed) in column 1
- Product 3 (fourth product, 0-indexed) in column 2
- Product 2 (third product, 0-indexed) in column 3

Important: Product numbers in the URL are **1-based** (starting from 1), but they are converted to 0-based indexes internally.

URL Parameter Examples

The `apple-compare` parameter overrides which products appear in specific column positions, from left to right. You only need to specify the columns you want to change.

Change the first column only:

```
?apple-compare=3
```

Shows product 3 (index 2) in the first column, other columns remain at their default products

Change the first two columns:

```
?apple-compare=1,3
```

Shows products 1 and 3 (indexes 0 and 2) in the first two columns, remaining columns use defaults

Change all columns (for a 3-column compare):

```
?apple-compare=4,3,2
```

Shows products at indexes 3, 2, and 1 in that order

How URL Parameters Work

1. The URL parameter overrides the `initialColumnState` from the JSON file
2. Parameters apply to **all viewports** (small, medium, large)
3. Invalid product numbers are filtered out
4. Duplicate numbers are removed automatically

Requirements for URL Parameters

For URL parameters to work, the compare must be configured as "configurable". This is controlled in the HTML's `data-channel-html-compare` attribute:

```
data-channel-html-compare='{"isConfigurableCompare": true, ...}'
```

If `isConfigurableCompare` is `false`, URL parameters will be ignored.

`data-compare-lineup` Attribute Configuration

The `data-compare-lineup` attribute provides an alternative way to configure the product lineup directly in the HTML file, without requiring URL parameters.

How to Add the Attribute

1. Open `index.html` in your text editor
2. Find the main compare container element (usually `<div class="channel-compare">`)
3. Add the `data-compare-lineup` attribute with your product configuration

Example:

```
<div class="channel-compare"
     data-channel-html-compare='{"isConfigurableCompare": true, ...}'
     data-compare-lineup='{"apple-compare": ["1","3","4"]}'>
```

Attribute Format

The attribute value must be valid JSON with this structure:

```
{"apple-compare": ["1", "2", "3"]}
```

Examples

Change the first column only:

```
data-compare-lineup='{"apple-compare": ["3"]}'
```

Shows product 3 (index 2) in the first column, other columns remain at their default products

Change the first two columns:

```
data-compare-lineup='{"apple-compare": ["1", "3"]}'
```

Shows products 1 and 3 (indexes 0 and 2) in the first two columns, remaining columns use defaults

Change all columns (for a 3-column compare):

```
data-compare-lineup='{"apple-compare": ["4", "3", "2"]}'
```

Shows products at indexes 3, 2, and 1 in that order

Requirements

For the `data-compare-lineup` attribute to work:

1. **`isConfigurableCompare` must be `true`** in the `data-channel-html-compare` attribute:

   ```
   data-channel-html-compare='{"isConfigurableCompare": true, ...}'
   ```

2. **Valid JSON format** - Use a JSON validator if you encounter issues
3. **Product numbers must exist** - Reference only products that exist in your JSON file

Priority Order

When multiple configuration sources exist, they are applied in this order (highest to lowest priority):

1. **URL parameter** (`?apple-compare=1,2,3`) - Highest priority
2. **data-compare-lineup attribute** (if present in HTML)
3. **initialColumnState from JSON** - Default/fallback